T.C.
TOKAT GAZİOSMANPAŞA ÜNİVERSİTESİ
BİLGİSAYAR TEKNOLOJİLERİ BÖLÜMÜ

MESLEKİ ÇÖZÜMLEME DERSİ
5. HAFTA LABORATUVAR RAPORU
(Veri Tabanı Uygulaması ve Geliştirme)

Proje / Senaryo Adı	Cüzdanım - Kişisel Bütçe Takip Uygulaması
Öğrenci Adı Soyadı	Bünyamin Burak Ulaşlı
Öğrenci Numarası	254309049
GitHub Depo (Nihai Sürüm) Linki [ZORUNLU]	https://github.com/BunyaminBurakUlasli/C-zdan-m-Ki-isel-B-t-e-Takip-Sistemi-.git

1. Projenin Amacı ve Haftalık Hedefler

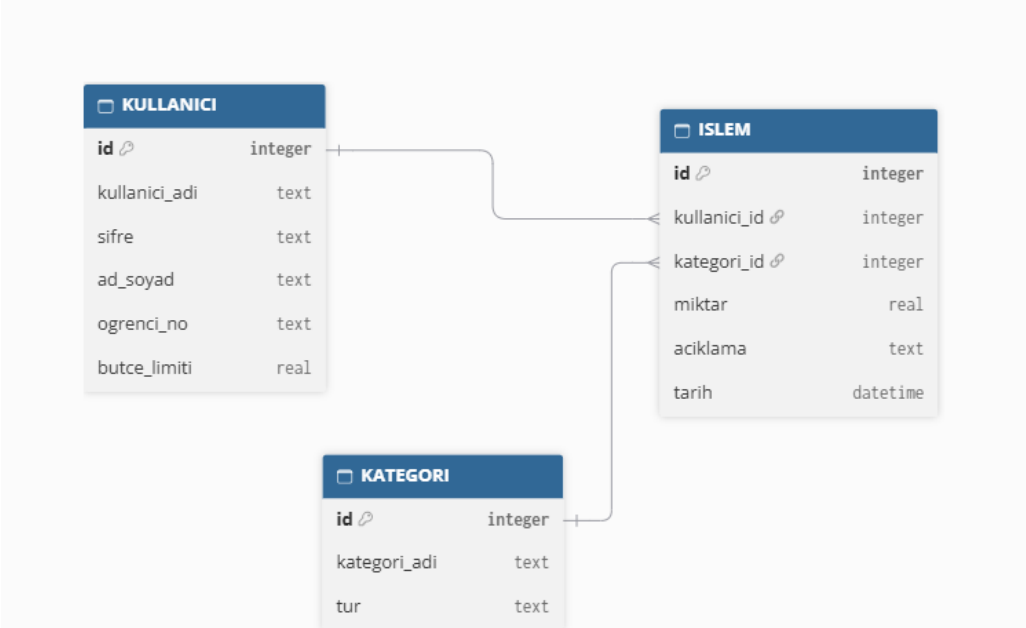
Kişisel Bütçe Takip Uygulaması (Cüzdanım) projemizin bu haftaki temel amacı, geçtiğimiz hafta kavramsal tasarımı tamamlanan Varlık-İlişki (ER) diyagramını, SQL dili kullanılarak fiziksel bir veri tabanı mimarisine dönüştürmektir. Bu kapsamda, projenin yapısına uygun olarak hafif ve ilişkisel bir yapı sunan SQLite sistemi tercih edilmiştir.

2. Yapılan Çalışmaların Detaylı Açıklaması (ZORUNLU ALAN)

Bu hafta DB Browser for SQLite ; programı kullanılarak uygulamanın arka planında çalışacak olan “cuzdanim.db” veri tabanı dosyası oluşturulmuştur. Çalışma adımları ve teknik kararlar şu şekildedir:

1. Tablo Oluşturma Sırası (İlişkisel Bütünlük): Foreign Key (Yabancı Anahtar): bağlantılarında hata almamak için, önce hiçbir tabloya bağımlı olmayan “KULLANICI” ve “KATEGORİ” tabloları SQL sorgularıyla oluşturulmuştur. En son bu iki tabloyu birbirine bağlayan “İŞLEM” tablosu kodlanmıştır.
2. Veri Tipi Seçimleri: Finansal doğruluk gerektiren miktar ve bakiye sütunları için ondalıklı sayıları destekleyen “REAL” veri tipi kullanılmıştır. Tarih/Saat işlemleri için “DATETIME” isim ve açıklamalar için ise “TEXT” tipi tercih edilmiştir.
3. Anahtar Atamaları: Her tabloya benzersiz bir kimlik kazandırmak için ID sütunları “INTEGER PRIMARY KEY AUTOINCREMENT” (Otomatik artan birincil anahtar) olarak tanımlanmıştır. Tablolar arası 1:N ilişkiler “FOREIGN KEY” kısıtlamaları ile güvence altına alınmıştır.

3. Veri Tabanı Şeması ve Tablo Yapıları



Tablo	Birincil Anahtar (PK)	Yabancı Anahtarlar (FK)
KULLANICI	id	—
KATEGORİ	id	—
İŞLEM	id	kullanici_id -> KULLANICI, kategori_id -> KATEGORİ

4. Örnek SQL Sorguları / Kod Parçacıkları

1. Tablo Oluşturma (CREATE TABLE) Sorgusu:

```
CREATE TABLE Islemler (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    kullanici_id INTEGER NOT NULL,
    kategori_id INTEGER NOT NULL,
```

```
    miktar REAL NOT NULL,
    aciklama TEXT,
    tarih DATETIME NOT NULL,
    FOREIGN KEY (kullanici_id) REFERENCES Kullanicilar (id),
    FOREIGN KEY (kategori_id) REFERENCES Kategoriler (id)
);
```

2. Veri Ekleme (INSERT INTO) Sorgusu:

```
INSERT INTO Islemler (kullanici_id, kategori_id, miktar, aciklama, tarih)
VALUES (1, 4, 450.00, 'Aylık internet faturası', '2026-10-19 14:30:00');
```

3. Veri Listeleme ve Birleştirme (SELECT & INNER JOIN) Sorgusu:

```
SELECT Islemler.id, Islemler.tarih, Kategoriler.kategori_adi, Kategoriler.tur, Islemler.miktar,
Islemler.aciklama
FROM Islemler
INNER JOIN Kategoriler ON Islemler.kategori_id = Kategoriler.id
ORDER BY Islemler.tarih DESC;
```

5. Karşılaşılan Zorluklar ve Çözümler

Tasarım aşamasından kodlama aşamasına geçerken en çok tabloları birbirine bağlarken (Foreign Key) zorluk yaşadım. Hata almamın temel sebebi, alt tablo olan İŞLEM tablosunu oluştururken henüz KULLANICI ve KATEGORİ tablolarının sistemde var olmamasıydı. Veri tabanı, henüz oluşturulmamış tablolara bağlanmaya çalıştığı için hata veriyordu.

Bu sorunu kodların çalışma sırasını değiştirerek çözdüm. Önce hiçbir yere bağımlı olmayan KULLANICI ve KATEGORİ tablolarını oluşturdum, en son aşamada ise bu iki tabloyu birbirine bağlayan İŞLEM tablosunu ekledim. Böylece tablolar arası bağlantı sorunları giderildi ve veri tabanı hatasız şekilde çalıştı.

Öğrenci İmzası:

